WhistApp: an In-Terminal Card Game

Software-ontwerp
2025-2026

Iteration 1

Contents

1 Introduction **2**

2 General Information **2**
2.1 Team Work 2
2.2 Iterations 3
2.3 The Software 3
2.4 Testing 3
2.5 UML Tools 4
2.6 What You Should Hand In 4
2.6.1 Late Submission Policy 4
2.6.2 When Toledo Fails 5
2.7 Evaluation 5
2.7.1 Presentation Of The Current Iteration 5
2.8 Peer/Self-assessment 6
2.9 Deadlines 6

3 WhistApp **6**
3.1 Whist 7
3.2 Bids 7
3.2.1 Points 8
3.3 Domain Model 9

4 Use Cases **9**
4.1 Use Case: Start new count 10
4.2 Use Case: In-app game 11
4.3 Use Case: View last-played trick 12

5 Implementation **12**

1 Introduction

For the course *Software-ontwerp*, you will design and develop *WhistApp*, an in-terminal card game. The main challenge will be to simulate the game's rules and play the game in the application. In Section 2, we explain how the project is organized, discuss the quality requirements for the software you will design and develop, and describe how we evaluate the solutions. In Section 3, we explain the problem domain of the application. The use cases are discussed in Section 4. Finally, we specify some implementation constraints in Section 5.

2 General Information

In this section, we explain how the project is organized, what is expected of the software you will develop and the deliverables you will hand in.

2.1 Team Work

For this project, you will work in groups of four. Each group is assigned an advisor from the educational staff. If you have any questions regarding the project, you can contact your advisor and schedule a meeting. When you come to the meeting, you are expected to prepare specific questions and have sufficient design documentation available. *If the design documentation is not of sufficient quality, the corresponding question will not be answered.* It is your own responsibility to organize meetings with your advisor and we advise to do this regularly. Experience from previous years shows that groups that regularly meet with their advisors produce a higher quality design. If there are problems within the group, you should immediately notify your advisor. Do not wait until right before the deadline or the exam!

To ensure that every team member practices all topics of the course, a number of roles are assigned by the team itself to the different members at the start of each iteration (or shortly thereafter in case of the first iteration). A team member that is assigned a certain role will give the presentation or demo corresponding to that role at the end of the iteration. That team member is *not supposed to do all of the work concerning his task*! He must, however, take a coordinating role in that activity (dividing the work, sending reminders about tasks to be done, make sure everything comes together, etc.), and be able to answer most questions on that topic during the evaluation. The following roles will be assigned round-robin:

Design Coordinator The design coordinator coordinates making the design of your software.

Testing Coordinator The testing coordinator coordinates the planning, designing, and writing of the tests for the software.

Domain Coordinator The domain coordinator coordinates the maintenance of the domain model.

As already mentioned, the goal of these roles is to make every team member participate in all aspects of the development of your system. **During each presentation or demo, every team member must be able to explain the**

used domain model, the design of the system, and the functioning of your test suite.

2.2 Iterations

The project is divided into 3 iterations. In the first iteration, you will implement the base functionality of the software. In subsequent iterations, new functionality will be added and/or existing functionality will be changed.

2.3 The Software

The focus of this course is on the *quality* (maintainability, extensibility, stability, readability,. . .) of the software you write. We expect you to use the development process and the techniques that are taught in this course. One of the most important concepts are the General Responsibility Assignment Software Principles (GRASP). These allow you to talk and reason about an object oriented design. **You should be able to explain all your design decisions in terms of GRASP**.

You are required to provide class and method documentation as taught in previous courses (e.g. the OGP course), as appropriate. When designing and implementing your system, you should use a *defensive programming style*. This means that the *client* of the public interface of a class cannot bring the objects of that class, or objects of connected classes, into an inconsistent state.

Unless explicitly stated in the assignment, you do not have to take into account persistent storage, security, multi-threading, and networking. If you have doubts about other non-functional concerns, please ask your advisor.

2.4 Testing

All functionality of the software should be tested. **For each use case, there should be a dedicated scenario test class.** For each use case flow, there should be at least one test method that tests the flow. Make sure you group your test code per step in the use case flow, indicating the step in comments (e.g. `// Step 4b`). Scenario tests should not only cover success scenarios, but also negative scenarios, i.e., whether illegal input is handled defensively and exceptions are thrown as documented. You determine to which extent you use unit testing. The testing coordinator briefly motivates the choice during the evaluation of the iteration.

Tests should have good coverage, i.e. a testing strategy that leaves large portions of a software system untested is of low value. If your IDE's test coverage functionality reports that only 60% of your code is covered by tests, this indicates there may be a serious problem with (the execution of) your testing strategy. However, be careful when drawing conclusions from both reported high coverage and reported low coverage (and understand why you should be careful). The testing coordinator is expected to briefly report the results of running the test suite with coverage tracking during the evaluation of the iteration.

2.5 UML Tools

There are many tools available to create UML diagrams depicting your design. You are free to use any of these as long as it produces correct UML. One of these UML tools is Visual Paradigm. You can find a link to a website where you can download the program and find the license key. However, we recommend UMLet for its simplicity.

2.6 What You Should Hand In

Exactly *one* person of the team hands in a ZIP-archive via Toledo. The archive contains the items below and follows the structure defined below. **Make sure that you use the prescribed directory names.**

- directory `groupXX` (where XX is your group number (e.g. 01, 12, ...))
  - `doc`: a folder containing, for each API you define in your system, a subfolder with the Javadoc for that API. For example, if you implemented a library of GUI components, include a subfolder with the Javadoc for this library's API.
  - `diagrams`: a folder containing UML diagrams that describe your design (at least one structural overview of your entire design, and sufficient detailed structural and behavioural diagrams to illustrate every use case)
  - `src`: your system's source code
  - `system.jar`: your system's compiled executable
  - `design.pdf` (optional): a document that clarifies your design and the main decisions; this document can be a prose text or a slide deck, or any other form that you deem appropriate.

When including your source code into the archive, make sure to *not include files from your version control system*. Make sure you choose relevant file names for your analysis and design diagrams (e.g. `SSDsomeOperation.png`). You do **not** have to include the project file of your UML tool, only the exported diagrams. We should be able to start your system by executing the JAR file with the following command: `java -jar system.jar`. If more setup is needed, include a README document with instructions.

Needless to say, the general rule that anything submitted by a student or group of students must have been authored exclusively by that student or group of students, and that accepting help from third parties constitutes exam fraud, applies here.

2.6.1 Late Submission Policy

If the zip file is submitted N minutes late, with $0 \leq N \leq 240$, the score for all team members is scaled by $(240 - N)/240$ for that iteration. For example, if your solution is submitted 30 minutes late, the score is scaled by 87.5%. So the maximum score for an iteration for which you can earn 4 points is reduced to 3.5. If the zip file is submitted more than 4 hours late, the score for all team members is 0 for that iteration.

2.6.2 When Toledo Fails

If the Toledo website is down – and *only* if Toledo is down – at the time of the deadline, submit your solution by e-mailing the ZIP-archive to your advisor. The timestamp of our e-mail server counts as your submission time.

2.7 Evaluation

After iteration 1, and again after iteration 2, there will be an intermediate evaluation of your solution. An intermediate evaluation lasts 35 minutes and consists of: a presentation about the design and the testing approach, accompanied by a demo of the system and the test suite.

The intermediate evaluation of an iteration will cover only the part of the software that was developed during that iteration. Before the final exam, the *entire* project will be evaluated. It is your own responsibility to process the feedback, and discuss the results with your advisor.

The evaluation of an iteration is planned in the week after that iteration. Immediately after the evaluation is done, you mail the PDF file of your presentation to Prof. Tom Holvoet and to your advisor.

2.7.1 Presentation Of The Current Iteration

The main part of the presentation should cover the design. The motivation of your design decisions *must* be explained in terms of GRASP principles. Use the appropriate design diagrams to illustrate how the most important parts of your software work. Your presentation should cover the following elements. Note that these are not necessarily all separate sections in the presentation.

1. A discussion of the high level design of the software (use GRASP patterns). Give a rationale for all the important design decisions your team has made.

2. A more detailed discussion of the parts that you think are the most interesting in terms of design (use GRASP patterns). Again we expect a rationale here for the important design decisions.

3. A discussion of the testing approach used in the current iteration.

4. An overview of the project management. Give an approximation of how many hours each team member worked. Use the following categories: group work, individual work, and study (excluding the classes and exercise sessions). In addition, insert a slide that describes the roles of the team members of the current iteration, and the roles for the next iteration. Note that these slides do not have to be presented, but we need the information.

Your presentation should not consist of slides filled with text, but of slides with clear design diagrams and keywords or a few short sentences. The goal of giving a presentation is to communicate a message, not to write a novel. All design diagrams should be *clearly readable* and use the correct UML notation. It is therefore typically a bad idea to create a single class diagram with all information. Instead, you can for example use an overview class diagram with only the most

important classes, and use more detailed class diagrams to document specific parts of the system. Similarly, use appropriate interaction diagrams to illustrate the working of the most important (or complex) parts of the system.

2.8 Peer/Self-assessment

In order for you to critically reflect upon the contribution of each team member, you are asked to perform a peer/self-assessment within your team. For each team member (including yourself) and for each of the criteria below, you must give a score on the following scale: *poor/lacking/adequate/good/excellent*. The criteria to be used are:

- Design skills (use of GRASP and DESIGN patterns, ...)
- Coding skills (correctness, defensive programming, documentation,...)
- Testing skills (approach, test suite, coverage, ...)
- Collaboration (teamwork, communication, commitment)

In addition to the scores themselves, we expect you to briefly explain for each of the criteria why you have given these particular scores to each of the team members. The total length of your evaluation should not exceed 1 page.

Please be fair and to the point. Your team members will not have access to your evaluation report. If the reports reveal significant problems, the project advisor may discuss these issues with you and/or your team. Please note that your score for this course will be based on the quality of the work that has been delivered, and not on how you are rated by your other team members.

Submit your peer/self-assessment by e-mail to both Prof. Tom Holvoet, your project advisor and TAs Sander Joos & Daan Vansteenhuyse, using the following subject: **[SWOP] peer-/self-assessment of group $groupnumber$ by $firstname$ $lastname$**.

2.9 Deadlines

- The deadline for handing in the ZIP-archive on Toledo is **11 March 2026, 3:30pm**.
- The deadline for submitting your peer/self-assessment is **15 March 2026, 6pm**, by e-mail to both your project advisor Prof. Tom Holvoet and TAs Sander Joos & Daan Vansteenhuyse.

3 WhistApp

The goal of the project is to develop an application you can use to keep the scoring for the Belgian variant of the game Whist (Wiezen in Dutch) and play it in the application, in case you forgot cards at home.

Whist is a trick taking game, where players each play a card from their hand and the highest card wins the trick (set of four played cards). Each round of the game consists of playing 13 trick between four players. Winning a round depends

on the bids that the players make before the round starts, and typically depend on winning a number of tricks or not winning any tricks at all.

For this course you'll design and implement a CLI based application that is able to take in, score and record any number of rounds played for a game of Whist.

The next section will explain the game of Whist in more detail. Section 4 will explain the use cases you need to implement.

3.1 Whist

This section explains a (simplified version of a) game of whist.

The game of Whist is played with a standard deck of 52 cards in four suits, and is dealt to four players in sets of 4, then 4, then 5 cards in clockwise fashion with the first set going the player next to the player dealing the cards. The last card dealt is turned over and shown to every player. The suit of the card determines the trump (troef in Dutch).

The value of a card depends on the suit and its value in the suit, ranked from high to low: A-K-Q-J-10-9-8-7-6-5-4-3-2. All suits are equal in value, except for the trump suit, whose cards trump all other cards from all other suits. If cards from the trump suit are included in a trick, the highest value trump card wins the trick, irrespective of the value of cards from the original leading suit (the player playing a trump 2 wins a hand containing an ace from another suit). If no trump cards were played, the highest card from the suit of the first played card wins.

A player is forced to play a card from the same suit as the first card that is played in a trick (colloquially called "following"). The suit of the first card may be the trump suit. If that player can't follow that suit, that player can play any card from another suit. This allows for playing a trump card in a trick that was not led with a trump card.

The player winning the trick is the one that has to play the first card for the next trick. For the first trick in a round, the player following the dealer is first to start, with some exceptions based on the bid.

3.2 Bids

Bidding starts from the player next to the dealer and proceeds clockwise. Any bid following a bid from a previous player should be higher than the preceding bids (with some exceptions). The bids in order of lowest to highest are:

- Pass: no bid is made by the player.
- Proposal: The player proposes to take 8 tricks with another player, with the trump suit being that of the last dealt card. If no player accepts (see the next bid) (e.g. all other players pass) the proposing player can choose to play on his own, needing 5 tricks to win. The player may also decide to still pass. See below for the rule on all players passing.
- Acceptance: the player will form a team with the player that proposed. Together, they need to win atleast 8 tricks. Extra points are awarded for each extra trick won, with points doubled if all 13 tricks are won.

- Abondance 9: the player will take 9 tricks on his own, may change the trump suit to one of his choosing and starts the game. He does not need to announce his choice of trump beforehand. This bid can be superseded by bidding the abondance 9 in the original trump suit.
- Abondance 10: Same as abondance 9, but 10 tricks to win.
- Miserie. The player will win no tricks in this round. The goal of the other 3 players is to force the player playing miserie into taking at least one trick. There is no longer a trump suit. This bid can be played by multiple players at the same time (if it is the highest bid of course).
- Abondance 11: Same as abondance 9, but 11 tricks to win.
- Abondance 12: Same as abondance 9, but 12 tricks to win.
- Open Miserie: The same as miserie, but the player of the miserie reveals his hand after the first trick is played. This bid can also be played by multiple players at the same time.
- Solo: Same as abondance 9, but 13 tricks to win.
- Solo slim: Same as solo, except you can not change the trump suit. You are forced to get all 13 tricks in the round, with the trump suit being that of the last dealt card.

If all players pass, the dealer stays the same and deals a new hand to each player (after the deck is cut again). The next round is **scored double**. If the cards need to be redealt twice, the scoring remains double, not quadruple.

3.2.1 Points

The points are based on the rules by IWWA and are also available in an overview table [1]. After each round, the total of the individual scores should always sum to zero, meaning if the winning players get points, the losing players get negative said points.

The rules for scoring are: if the bid is a two against two game, the points below are given to each of the winners and taken from each of the losers. If the game is one against three, the points are given to the winner and taken from the losers, where each of them pays a third of the points. Examples: the points for winning a solo slim bid is 90 points. If the player with the solo slim bid wins, he gains 90 points and the other three players lose 30 points each. if the solo slim player loses, he loses 90 points and the other players gain 30 points. If multiple players play the same bid (e.g. miserie), each bid is handled individually by this rule and the resulting losses/gains for each bid individually added to each other for the final scoring of the round.

The points for each of the bids are:

- Proposal:

[1] `https://5e98ae37-8a25-4c17-82c2-ca757ae9a98e.filesusr.com/ugd/e6fad6_3d866d012f68443a9b11bb133559333a.pdf`

  - On your own: 6 points + 3 for each excess trick (e.g. 6 tricks: 6 + 3 = 9).
  - With partner: 2 points + 1 for each excess trick (so, 9 tricks: 2 + 1 = 3)
  - All tricks: the above points are doubled.

- Abondance:
  - 9: 15 points (or -15 in case of a loss)
  - 10: 18 points
  - 11: 24 points
  - 12: 27 points

- Miserie: 21 points
- Open miserie: 42 points
- Solo: 75 points
- Solo slim: 90 points

These rules are one of the many variations used. You can read more about the game here [2] (this text was partially inspired by this link, especially for terminology) or here [3]. You can play the game yourself at `https://www.whisthub.com/wiezen`.

3.3 Domain Model

Figure 1 shows a domain model of WhistApp.

Note: the concepts, attributes, and associations in the domain model serve only to describe the required observable behavior of the system, considered as a black box. The domain model that is provided does not explain every technical aspect of the application, such as the fact that the scores for the players after each round need to be calculated based on their bids and their tricks. A large focus of this assignment is on the domain layer of the application.

Nothing about the domain model should be interpreted as statements about the internal structure of the system. Of course, in the same way that it is likely that an object-oriented implementation of a point-of-sale system will involve a class that is reminiscent of the ProductDescription concept from the domain model, it is likely that an object-oriented implementation of WhistApp will involve a class that is reminiscent of the Round concept. In fact, by the *Low Representational Gap* principle, this is generally considered recommended practice.

4 Use Cases

Figure 2 shows the use case diagram for WhistApp. The remainder of this section describes the use cases in detail.

[2] https://www.pagat.com/boston/wiezen.html
[3] `https://www.whistiwwa.com/reglementen-tornooien`

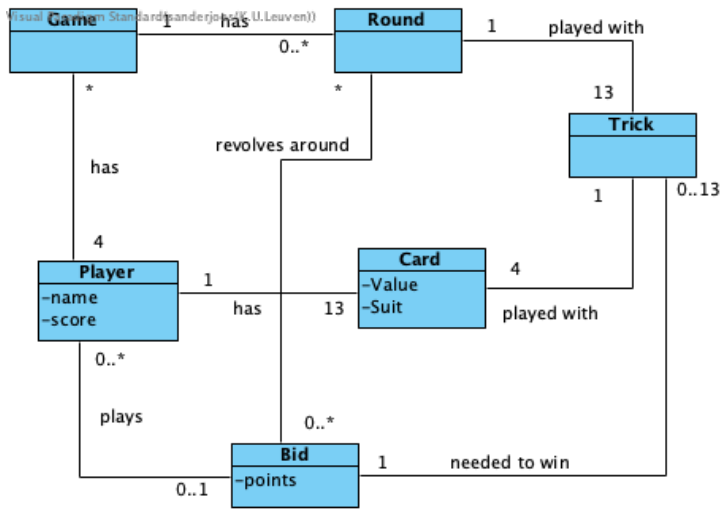

Figure 1: Domain model

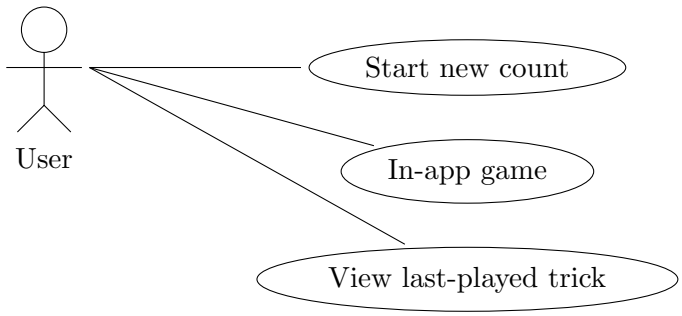

Figure 2: Use case diagram for WhistApp.

Note:

- A system's requirements can be specified at various levels of abstraction with respect to the nature of the interface between the system and its environment (e.g. a human user). Often the specification abstracts over the nature of the interface to focus on the aspects that are most relevant to the usefulness of the system.

4.1 Use Case: Start new count

Main Success Scenario:

1. The user selects to start a new count.
2. The system asks to register the names of the four players.
3. The system starts a new game with the registered players, with a score of zero for all players.

4. The user registers which bid will be played in the current round and the trump suit. The supported bids are Proposal (alone & with partner), Abondance 9-12, (Open) Miserie and Solo (slim).
5. The user registers which player(s) play the bid.
6. The user can enter the result of the round. This is either:
   (a) The user can enter which player(s) won and failed their bid (for (Open) Miserie).
   (b) The user can enter how many tricks the player(s) of the registered bid won (all other bids).
7. The system calculates the points for the round for all players and updates their scores.
8. The user selects to quit the game.

Extensions:

8a. The user restarts the game at 3., ensuring the scores are kept.

4.2 Use Case: In-app game

Main Success Scenario:

1. The user selects to start a new in-app game.
2. The system asks to register the names of the four players.
3. The system starts a new game with the registered players, with a score of zero for all players.
4. The system distributes the deck of cards randomly among all players, acting as the dealer, which is a randomly chosen player. The last card dealt is the trump card and is shown to all players.
5. Starting with the player left of the dealer, the player chooses a bid from all legal bids (e.g. Player 1 chooses *Abondance* 9, Player 2 can't select *Propose* since it has a lower priority). During bidding, each player can only see their own cards and the trump card. If all players made their bid, the game starts with the chosen bid.
6. The player who won the last bid, or the player left of the dealer, or in case of *abondance* the player with that bid if it's the first turn, plays a card, immediately ending their turn.
7. The game asks confirmation, before the cards of the next player are shown.
8. The next player plays a card, all previously played cards in the trick are shown. The system prevents illegal card plays.
9. 7. and 8. repeat for all four players. The trick is awarded to the player who has played the highest card in the suit of the trick, unless a trump card has been played in which case the highest trump card wins.
10. The system repeats 6. - 9. until all 13 cards have been played.
11. The system calculates the points for each player.

12. The user selects to quit the game.

Extensions:

2a. The user selects to replace players with one of two bots: the High-bot or the Low-bot. The High-bot will always play the highest card it legally can play (effectively trying to always win the trick). The Low-bot will always play the lowest card it can legally play (effectively trying to lose the trick). Both bots will pass during the bidding process. These bots can also be used to test your application by replacing all but one player with bots.

5a. If all players pass, cards are re-dealt and the use case starts again at 4, with the same dealer.

7a. Bots play automatically and do not need extra confirmation before their turn begins.

8a. In case the bid is Open Miserie, the cards of that player are always revealed.

12a. The user restarts the game at 4., ensuring scores are kept. The dealer is no longer random, but is moved to the left.

4.3 Use Case: View last-played trick

Precondition: An in-app game is still in progress and at least on trick has been played.

Main Success Scenario:

1. The user can request to view the cards played in the last trick.
2. The system shows each card played in the last trick and which player played each card.
3. The user plays a card and play continues as described in Use case 2 step 7.

5 Implementation

You have to develop your system in Java. The use of other libraries, or implementing terminal I/O directly yourself, are not allowed. Notice that this means you cannot use colors, fonts, or other special effects in your user interface. (The main purpose of this restriction is to keep you focused on the software design work and prevent you from wasting time with technical work that is not relevant to the course objectives.) With the use of System.out (for output) and the Java Scanner class (for input) you can build a functional application.

Good luck!
The SWOP Team members